

Backend API — Citizen Reporting Integration

For E (citizen-reporting frontend). Generated 2026-09-01. Backend: FastAPI + PostgreSQL/PostGIS, verified live end-to-end (real DB, real photo upload, real public URL) before this doc was written.

Base URL (rehearsal — read this first)

```
https://michael-suit-trial-shaped.trycloudflare.com
```

This URL is temporary. It's a free Cloudflare "quick tunnel" pointed at Sushanth's machine — it only works while his backend is running, and **a new URL is generated every time the tunnel restarts**. If this URL stops responding, ask Sushanth to re-run `scripts\run_public_dev_server.ps1` and send you the new one (printed at the end of that script, also in `bin\tunnel.log`).

Health check to confirm connectivity right now:

```
GET https://michael-suit-trial-shaped.trycloudflare.com/health
-> {"status": "ok"}
```

Endpoint & request format

POST `/reports` — no `/api/` prefix, just `<base-url>/reports`.

multipart/form-data, one request, two parts: - `data` — form field, value is a **JSON string** with your exact payload shape (below) - `photo` — the file itself, form field name `photo`, omit entirely if there's no photo

No auth header needed (open endpoint for this build). No required headers beyond what a browser sets automatically for multipart.

Field names — matched to yours exactly, no renaming needed

```
{
  "reportType": "crack | movement | road | other",
  "severity": "low | moderate | high | critical",
  "coords": { "lat": 27.xxxx, "lng": 88.xxxx, "accuracy": 15 },
  "placeName": "string, optional",
  "description": "string, min 5 characters",
  "reporterName": "string, optional",
  "reporterPhone": "string, optional",
  "capturedAt": "ISO 8601 timestamp"
}
```

This is the `data` field's value (as a JSON **string**, not a nested multipart field). `hasPhoto` isn't read by the backend — it just checks whether a `photo` file was actually attached, so include it or drop it, either is fine.

Real, working example

```
curl -X POST https://michael-suit-trial-shaped.trycloudflare.com/reports \
-F 'data={"reportType":"crack","severity":"moderate","coords":{"lat":27.3389,"lng":88.6065,"accuracy":15},"placeName":"Crack on road","description":"Crack on road","reporterName":"Sushanth","reporterPhone":"9876543210","capturedAt":"2026-09-01T12:00:00Z"}' \
-F 'photo=@photo.jpg;type=image/jpeg'
```

Photo handling

- **Max size: 10MB**
- **Accepted: JPEG, PNG, HEIC, HEIF** (checked via the upload's `Content-Type`)
- No server-side resizing/compression currently — compress client-side if your users are on weak connections, but it's not required for the request to succeed

Response

Success — **200**, full report object back:

```
{
  "id": "633368bc-66b0-4390-886a-357ae39609ff",
  "client_report_id": null,
  "zone_id": null,
  "report_type": "crack",
  "severity": "moderate",
  "geo_lat": 27.3389,
  "geo_lng": 88.6065,
  "geo_accuracy_m": 15.0,
  "place_name": "Near Ranipool bridge",
  "description": "Crack on retaining wall",
  "reporter_name": "Test",
  "reporter_phone": "9876543210",
  "photo_url": "/uploads/b1ad8331-1b61-4f44-8482-987b5063c739.jpg",
  "captured_at": "2026-09-01T16:00:00+05:30",
  "submitted_at": "2026-09-01T19:31:11.133343+05:30",
  "verified_status": "unverified"
}
```

`photo_url` is a relative path — prefix it with the base URL to actually view/display the photo (e.g. `<base-url>/uploads/b1ad8331-....jpg`).

Failure — **422**, structured error, not a generic message. Real examples, tested live:

```
// bad reportType
{"detail":[{"type":"literal_error","loc":["reportType"],"msg":"Input should be 'crack', 'movement', 'road' or

// description too short
{"detail":[{"type":"string_too_short","loc":["description"],"msg":"String should have at least 5 characters",

// wrong photo type
{"detail":"unsupported photo content type 'application/pdf', expected one of ['image/jpeg', 'image/png', 'image/
```

Auth & environment

- **No auth token** for this build — open endpoint
- **CORS**: enabled for all origins — your frontend's origin works regardless
- **Health check**: `GET /health` → `{"status": "ok"}`

Data contract

- **Report ID**: backend generates it (UUID). For offline queuing specifically: send an optional `client_report_id` (a UUID you generate client-side, before the request) in the data JSON. If you retry the same submission later (after reconnecting), the backend recognizes the same `client_report_id` and returns the existing report instead of creating a duplicate — safe to retry.
- **reporterName / reporterPhone**: genuinely optional, staying that way.
- **Validation to mirror client-side**: `description` ≥ 5 characters; `reportType` exactly one of the 4 listed values; `severity` exactly one of the 4 listed values — anything else gets rejected with a **422** before it's ever stored.

For the dashboard (D's job, useful to know)

`GET /reports` — returns the full list, newest first, JSON array of the same object shape as above. Polling, not websocket/push.

Restarting the stack (for Sushanth, if the URL dies)

```
powershell -ExecutionPolicy Bypass -File scripts\run_public_dev_server.ps1
```

Starts local Postgres+PostGIS, the backend, and a fresh public tunnel — prints the new URL at the end. To stop everything: `scripts\stop_public_dev_server.ps1`.